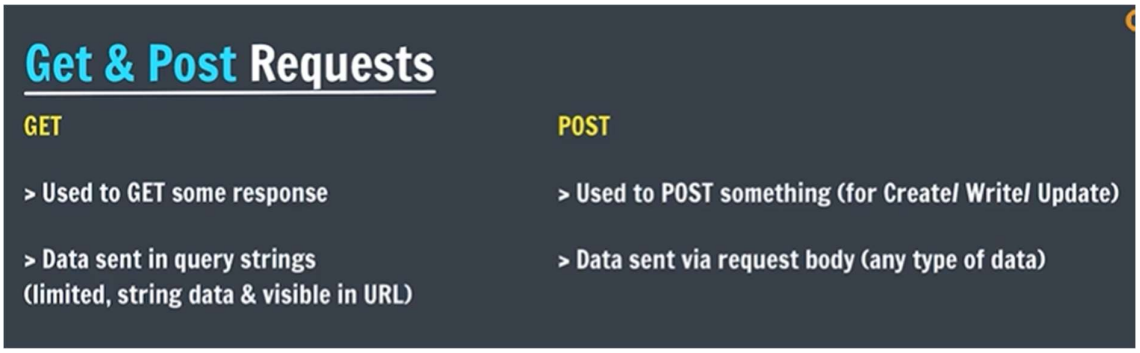
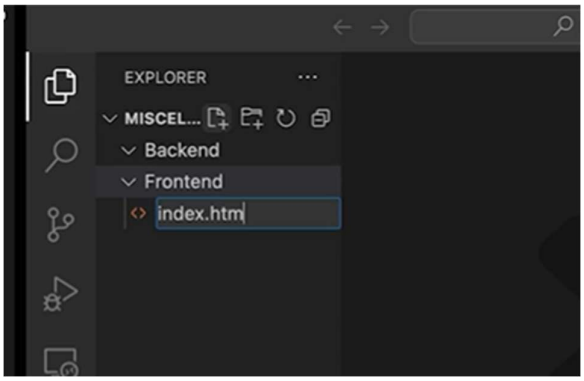
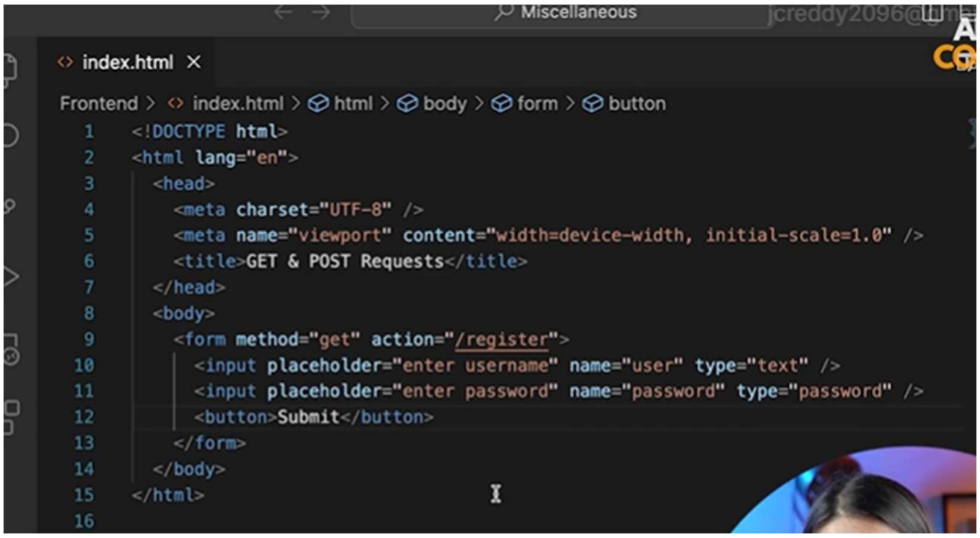
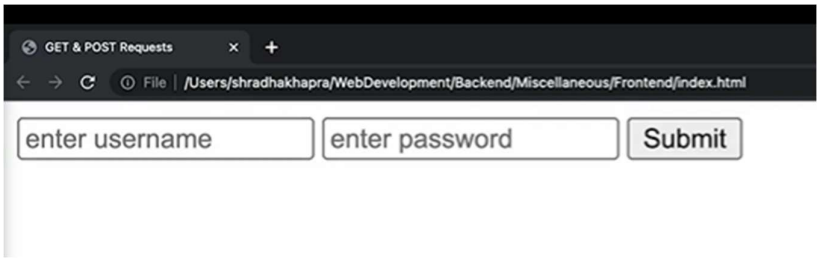
→Miscellenous

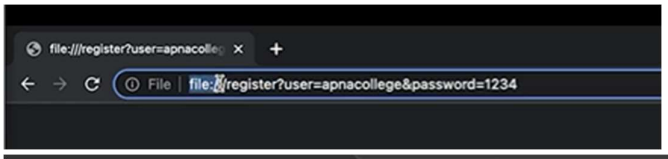


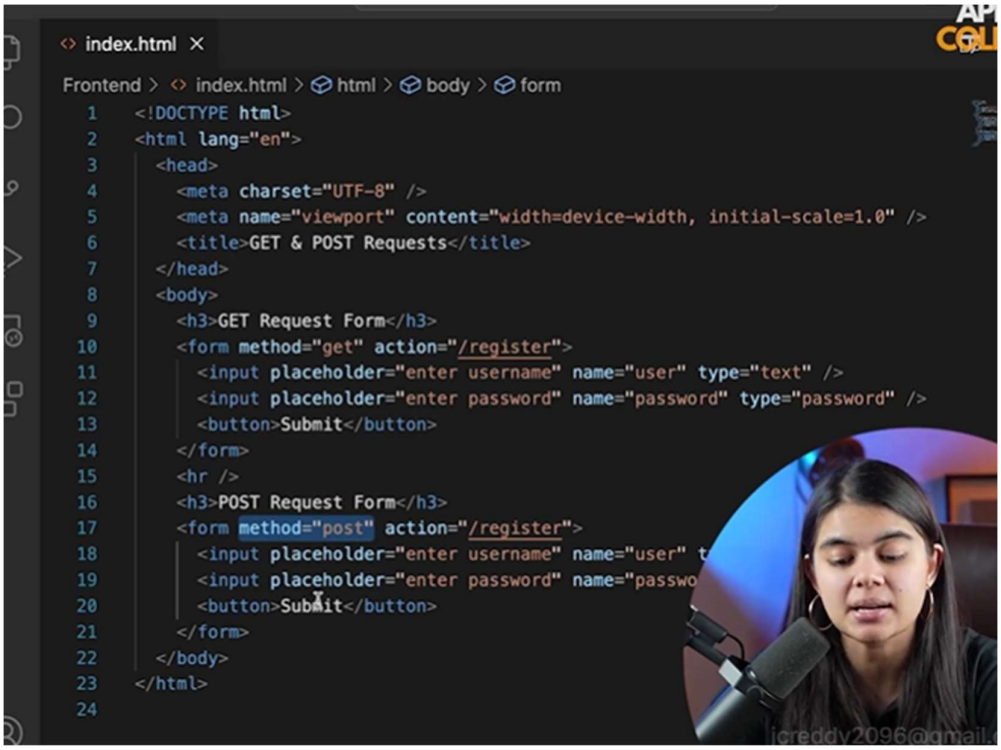


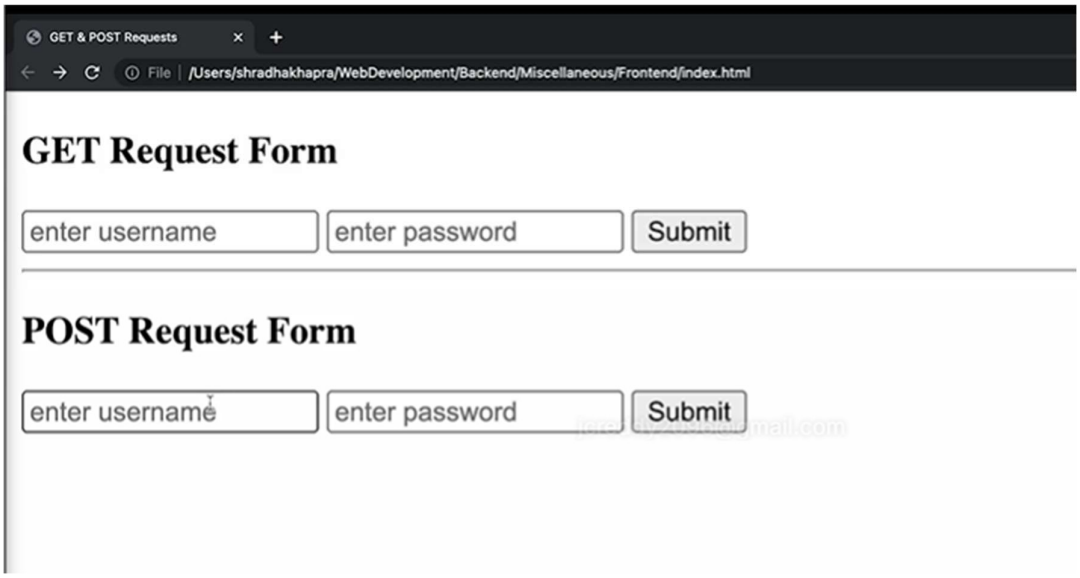


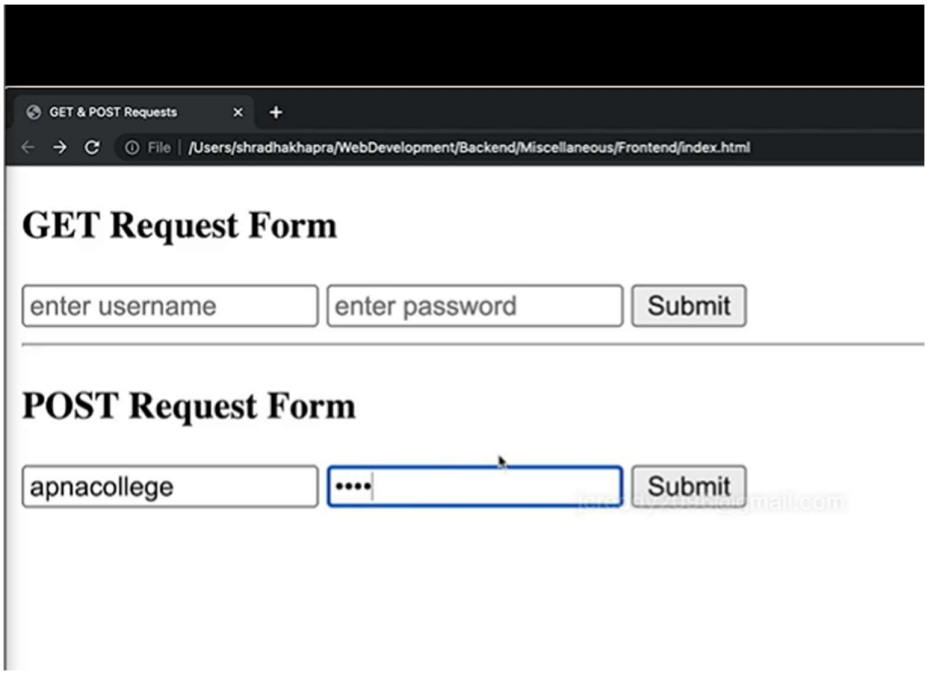


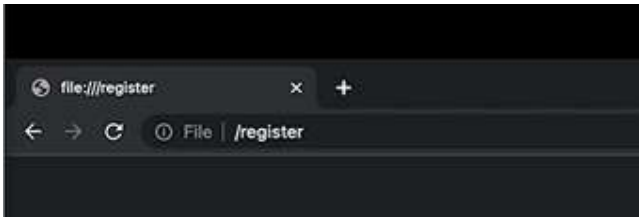

GET & POST Requests
File | /Users/shradhakhapra/WebDevelopment/Backend/Miscellaneous/Frontend/index.html
apnacollege
Submit

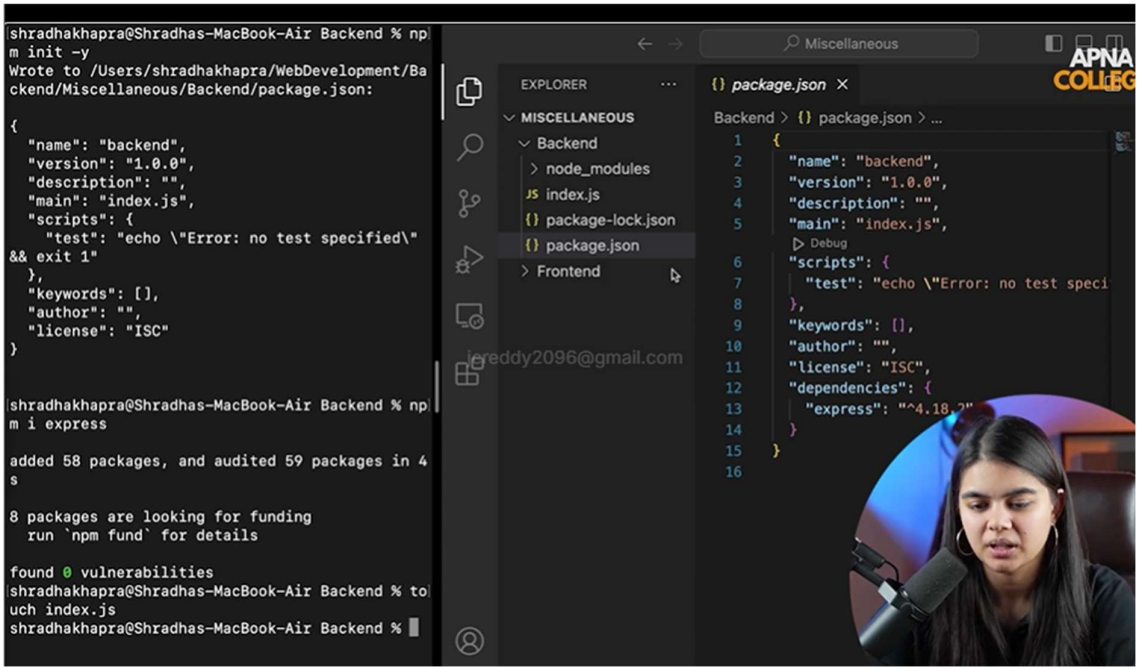

file:///register?user=apnacolleg
File | file:///register?user=apnacollege&password=1234

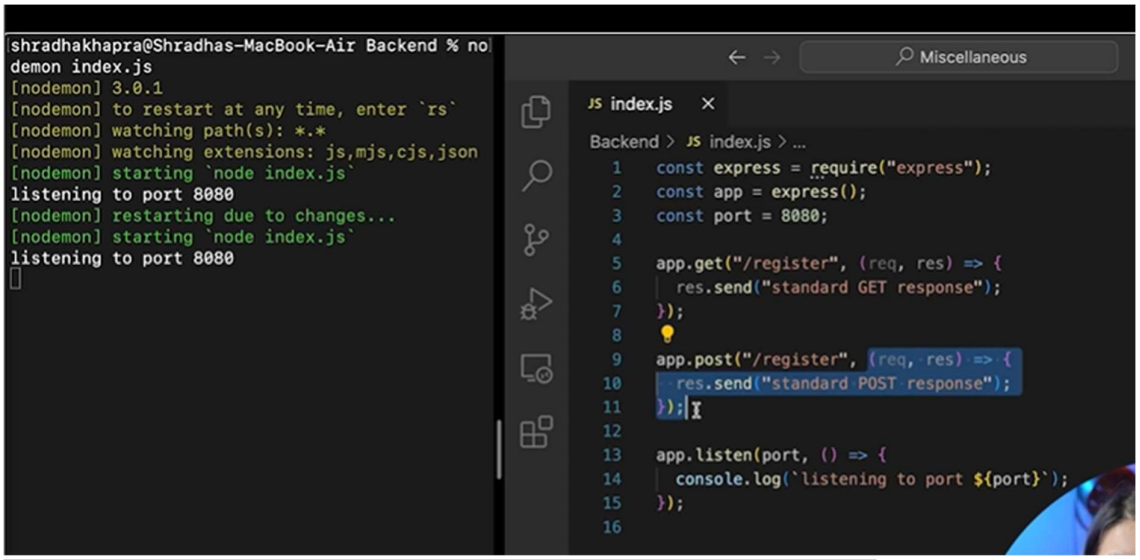

index.html
Frontend > index.html > html > body > form
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>GET & POST Requests</title>
</head>
<body>
<h3>GET Request Form</h3>
<form method="get" action="/register">
<input placeholder="enter username" name="user" type="text" />
<input placeholder="enter password" name="password" type="password" />
<button>Submit</button>
</form>
<hr />
<h3>POST Request Form</h3>
<form method="post" action="/register">
<input placeholder="enter username" name="user"
<input placeholder="enter password" name="passwo
<button>Submit</button>
</form>
</body>
</html>

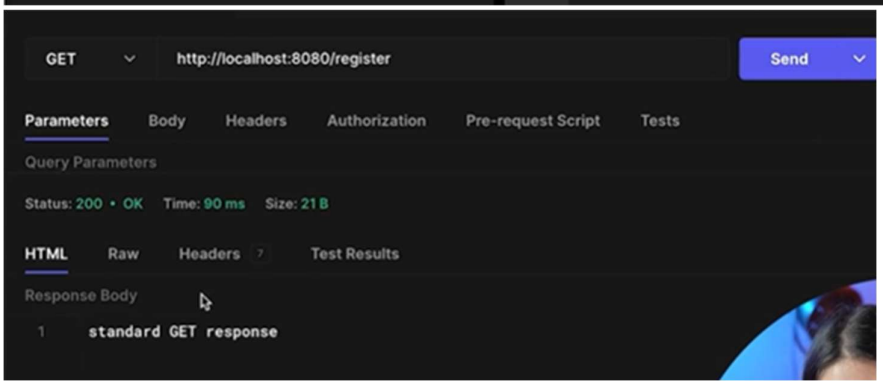

GET & POST Requests
File | /Users/shradhakhapra/WebDevelopment/Backend/Miscellaneous/Frontend/index.html
GET Request Form
enter username
enter password
Submit
POST Request Form
enter username
enter password
Submit

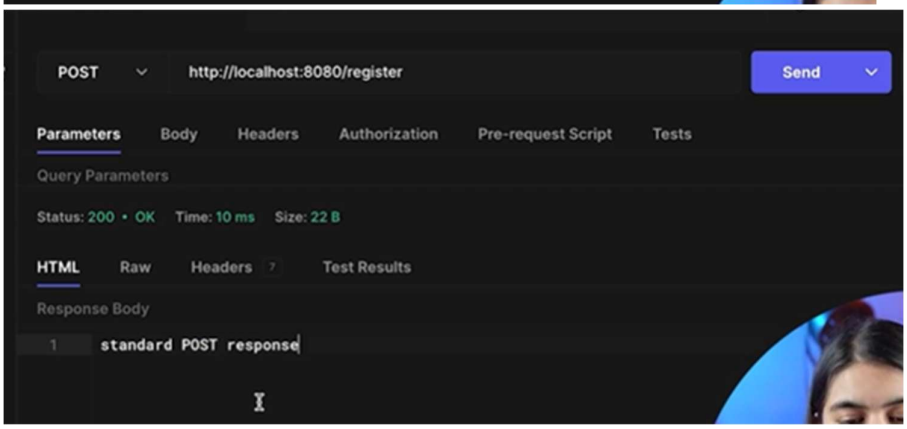

GET & POST Requests
File | /Users/shradhakhapra/WebDevelopment/Backend/Miscellaneous/Frontend/index.html
GET Request Form
enter username
enter password
Submit
POST Request Form
apnacollege
Submit


file:///register
File | /register


shradhakhapra@Shradhas-MacBook-Air Backend % npm init -y
Wrote to /Users/shradhakhapra/WebDevelopment/Backend/Miscellaneous/Backend/package.json:
{
"name": "backend",
"version": "1.0.0",
"description": "",
"main": "index.js",
"scripts": {
"test": "echo \"Error: no test specified\" && exit 1"
},
"keywords": [],
"author": "",
"license": "ISC"
}
shradhakhapra@Shradhas-MacBook-Air Backend % npm i express
added 58 packages, and audited 59 packages in 4s
8 packages are looking for funding
run `npm fund` for details
found 0 vulnerabilities
shradhakhapra@Shradhas-MacBook-Air Backend % touch index.js
shradhakhapra@Shradhas-MacBook-Air Backend %
Miscellaneous
EXPLORER
MISCELLANEOUS
Backend
node_modules
index.js
package-lock.json
package.json
Frontend
package.json
Backend > package.json > ...
"name": "backend",
"version": "1.0.0",
"description": "",
"main": "index.js",
Debug
"scripts": {
"test": "echo \"Error: no test speci
},
"keywords": [],
"author": "",
"license": "ISC",
"dependencies": {
"express": "^4.18.2"
}
}


shradhakhapra@Shradhas-MacBook-Air Backend % no
demon index.js
[nodemon] 3.0.1
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
listening to port 8080
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
listening to port 8080
Miscellaneous
index.js
Backend > index.js > ...
1 const express = require("express");
2 const app = express();
3 const port = 8080;
4
5 app.get("/register", (req, res) => {
6 res.send("standard GET response");
7 });
8
9 app.post("/register", (req, res) => {
10 res.send("standard POST response");
11 });
12
13 app.listen(port, () => {
14 console.log(`listening to port ${port}`);
15 });
16

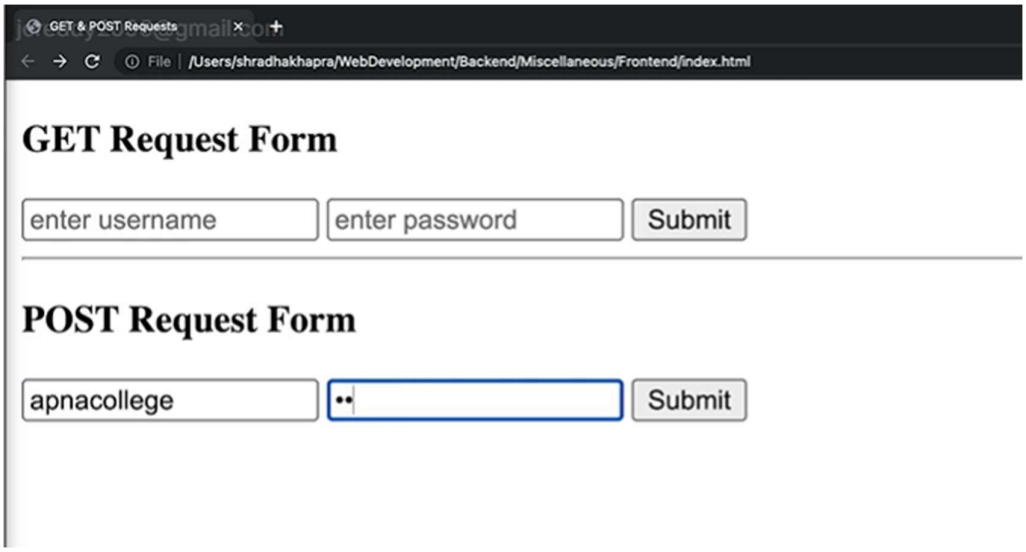

GET
http://localhost:8080/register
Send
Parameters
Body
Headers
Authorization
Pre-request Script
Tests
Query Parameters
Status: 200 • OK
Time: 90 ms
Size: 21 B
HTML
Raw
Headers
Test Results
Response Body
1 standard GET response

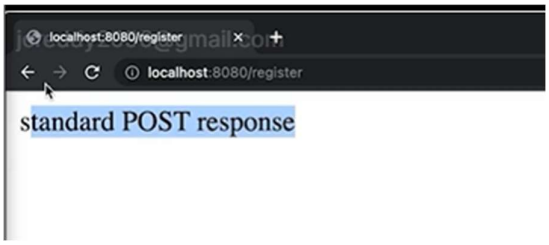

POST
http://localhost:8080/register
Send
Parameters
Body
Headers
Authorization
Pre-request Script
Tests
Query Parameters
Status: 200 • OK
Time: 10 ms
Size: 22 B
HTML
Raw
Headers
Test Results
Response Body
1 standard POST response

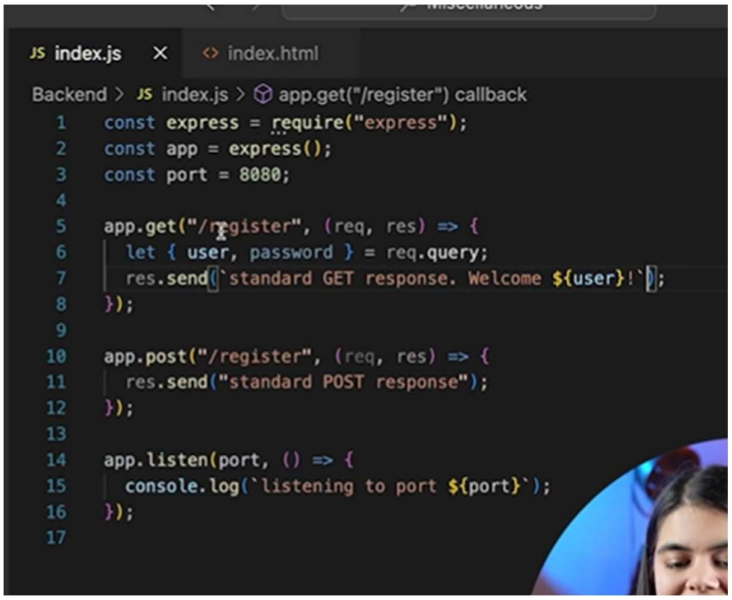
index.js | index.html

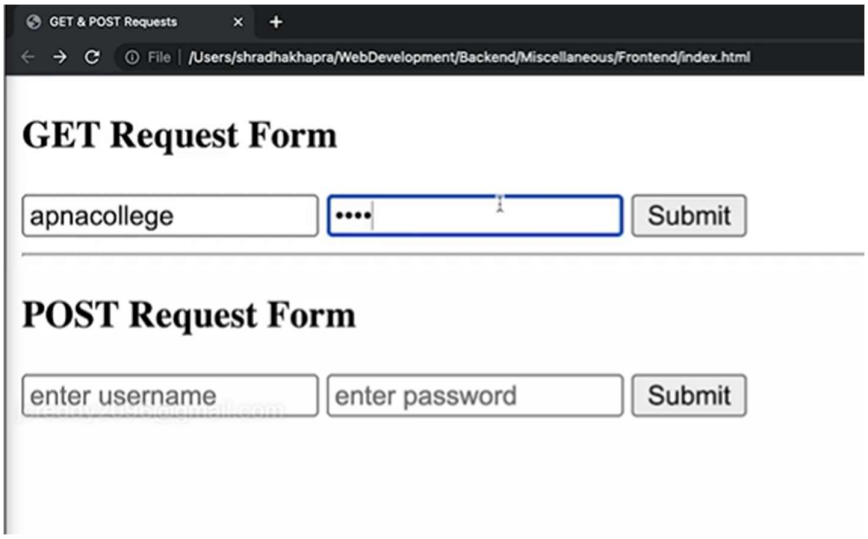
Frontend > index.html > html > body > form

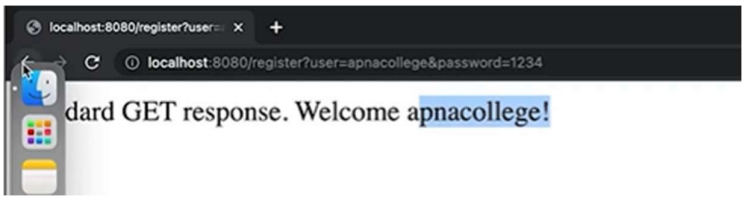
```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale
    <title>GET & POST Requests</title>
  </head>
  <body>
    <h3>GET Request Form</h3>
    <form method="get" action="http://localhost:8080/register">
      <input placeholder="enter username" name="user" type="text" />
      <input placeholder="enter password" name="password" type="pass
      <button>Submit</button>
    </form>
    <hr />
    <h3>POST Request Form</h3>
    <form method="post" action="http://localhost:8080/register">
      <input placeholder="enter username" name="user" type="text" />
      <input placeholder="enter password" name="password" type="pass
      <button>Submit</button>
    </form>
  </body>
</html>
```


GET & POST Requests
File | /Users/shradhakhapra/WebDevelopment/Backend/Miscellaneous/Frontend/index.html
GET Request Form
enter username
enter password
Submit
POST Request Form
apnacollege
Submit

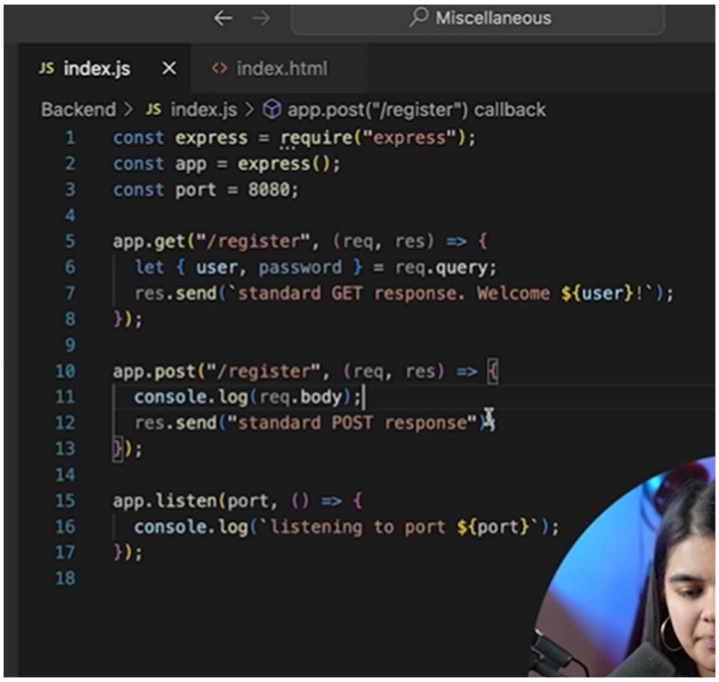

localhost:8080/register
localhost:8080/register
standard POST response

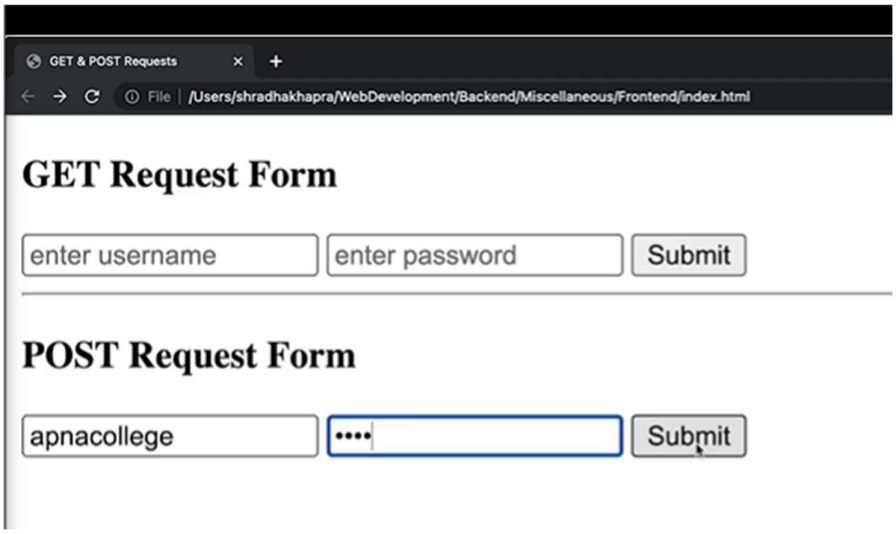

index.js
index.html
Backend > index.js > app.get("/register") callback
const express = require("express");
const app = express();
const port = 8080;
app.get("/register", (req, res) => {
let { user, password } = req.query;
res.send(`standard GET response. Welcome ${user}!`);
});
app.post("/register", (req, res) => {
res.send("standard POST response");
});
app.listen(port, () => {
console.log(`listening to port ${port}`);
});



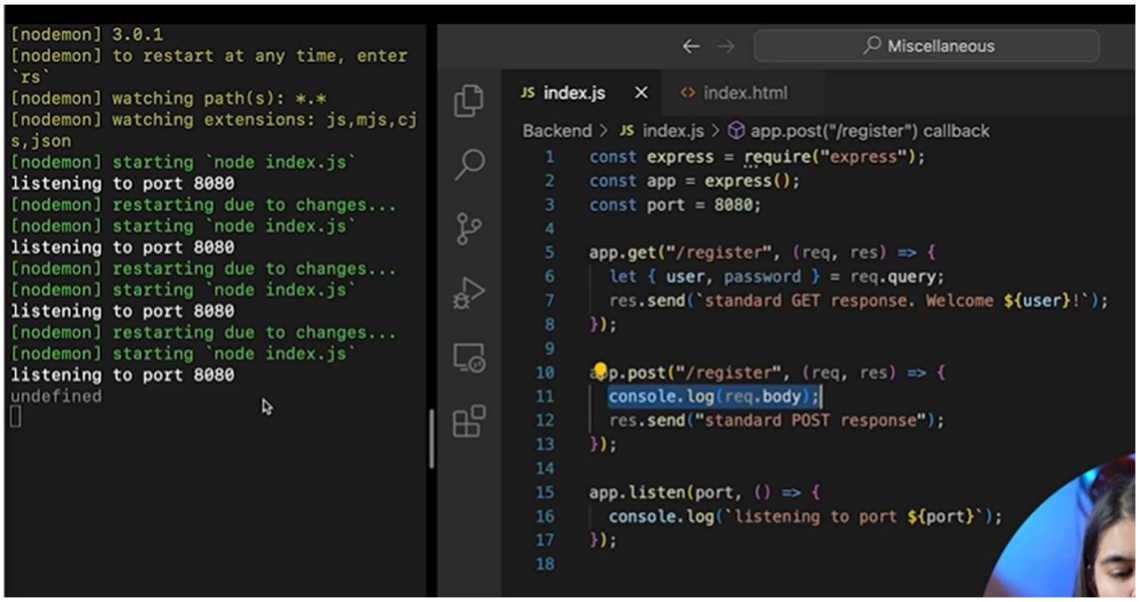


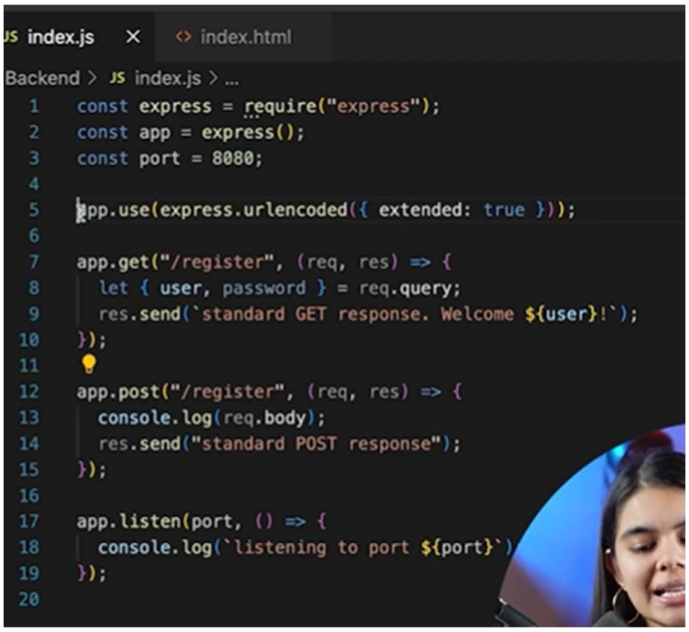
→Handling post requests


Miscellaneous
index.js
index.html
Backend > index.js > app.post("/register") callback
const express = require("express");
const app = express();
const port = 8080;
app.get("/register", (req, res) => {
let { user, password } = req.query;
res.send(`standard GET response. Welcome ${user}!`);
});
app.post("/register", (req, res) => {
console.log(req.body);
res.send("standard POST response");
});
app.listen(port, () => {
console.log(`listening to port ${port}`);
});

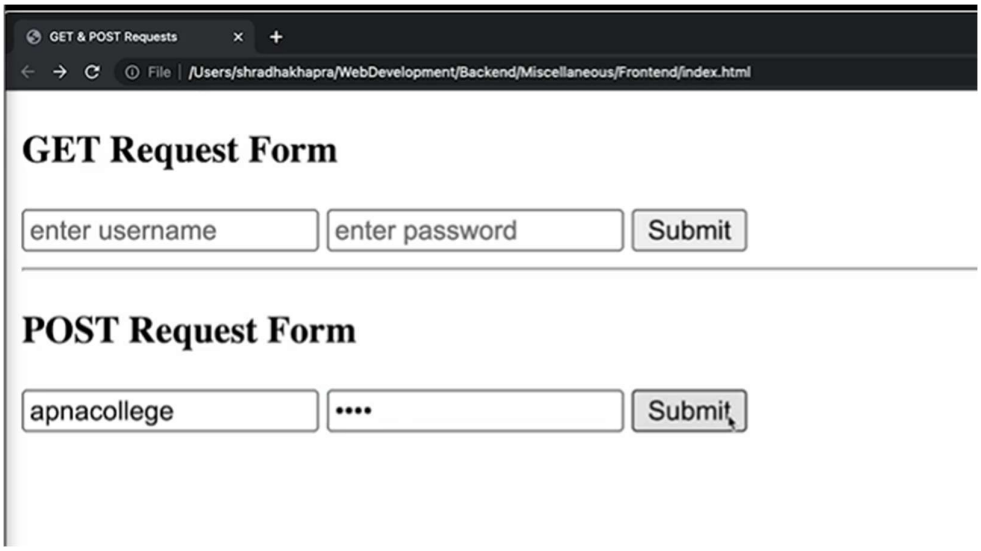

GET & POST Requests
File | /Users/shradhakhapra/WebDevelopment/Backend/Miscellaneous/Frontend/index.html
GET Request Form
enter username
enter password
Submit
POST Request Form
apnacollege
Submit

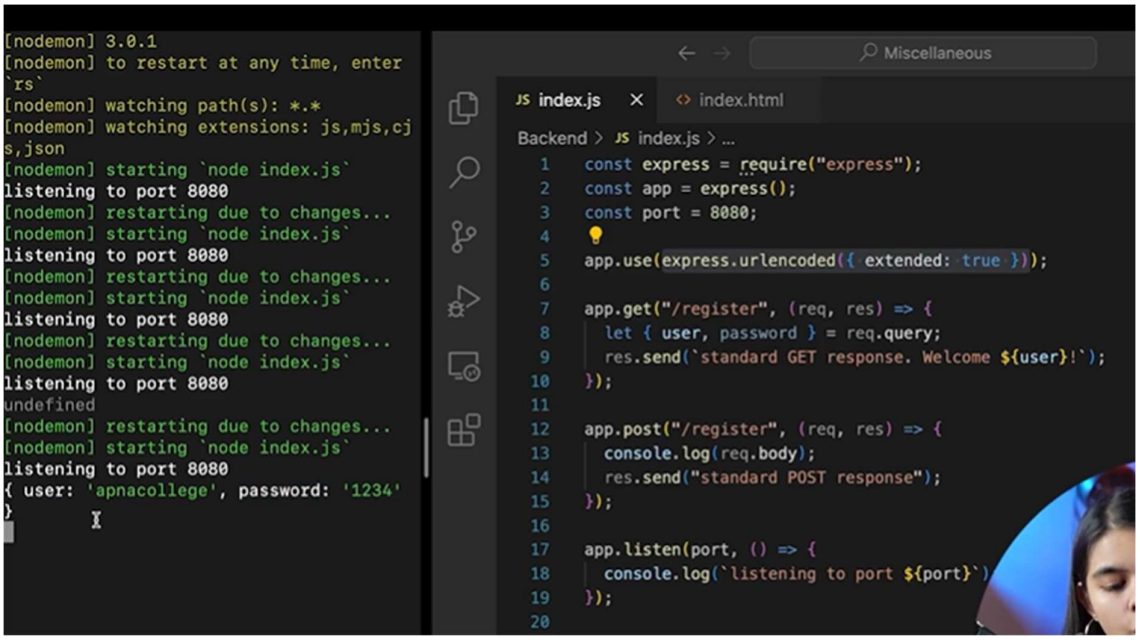

localhost:8080/register
localhost:8080/register
standard POST response

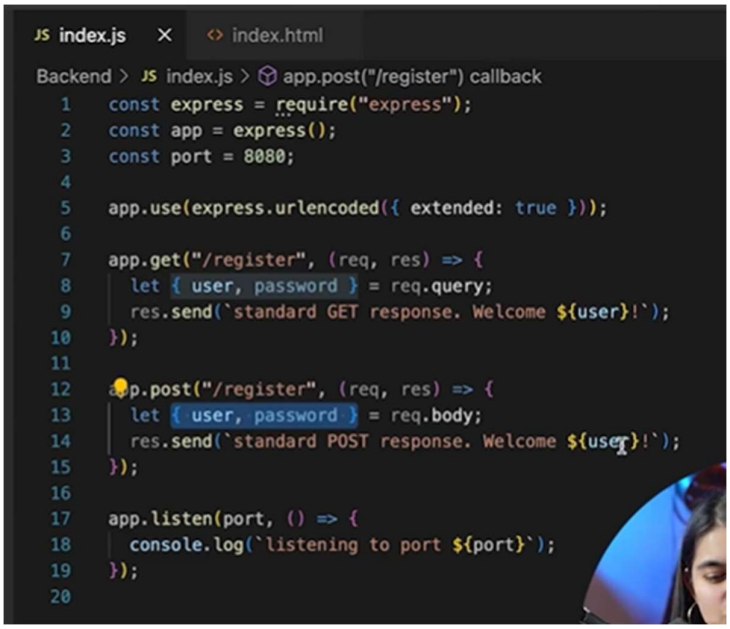

[nodemon] 3.0.1
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
listening to port 8080
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
listening to port 8080
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
listening to port 8080
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
listening to port 8080
undefined
Miscellaneous
index.js
index.html
Backend > index.js > app.post("/register") callback
const express = require("express");
const app = express();
const port = 8080;
app.get("/register", (req, res) => {
let { user, password } = req.query;
res.send(`standard GET response. Welcome ${user}!`);
});
app.post("/register", (req, res) => {
console.log(req.body);
res.send("standard POST response");
});
app.listen(port, () => {
console.log(`listening to port ${port}`);
});

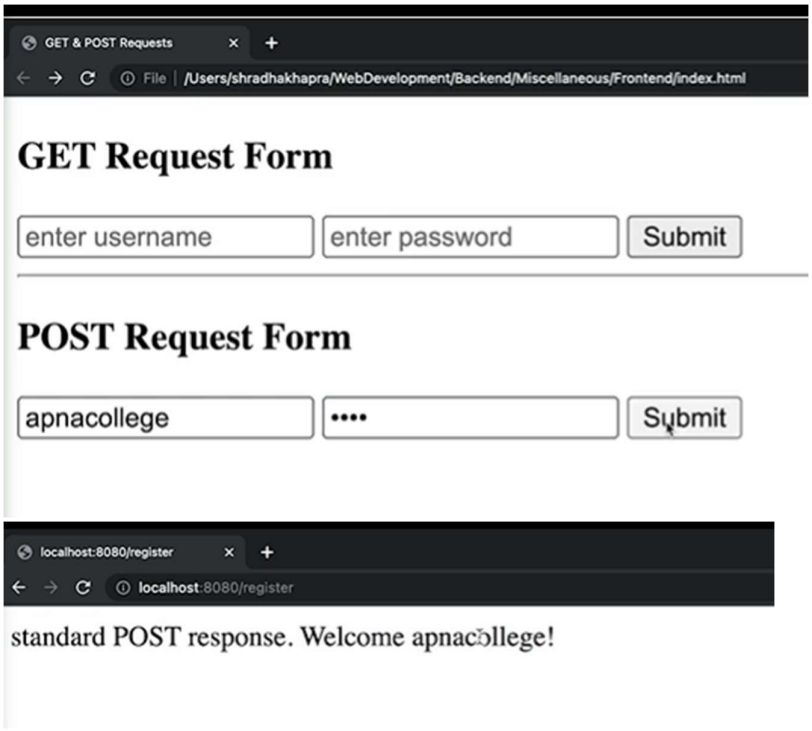

index.js
index.html
Backend > index.js > ...
const express = require("express");
const app = express();
const port = 8080;
app.use(express.urlencoded({ extended: true }));
app.get("/register", (req, res) => {
let { user, password } = req.query;
res.send(`standard GET response. Welcome ${user}!`);
});
app.post("/register", (req, res) => {
console.log(req.body);
res.send("standard POST response");
});
app.listen(port, () => {
console.log(`listening to port ${port}`)
});


localhost:8080/register
localhost:8080/register
standard POST response

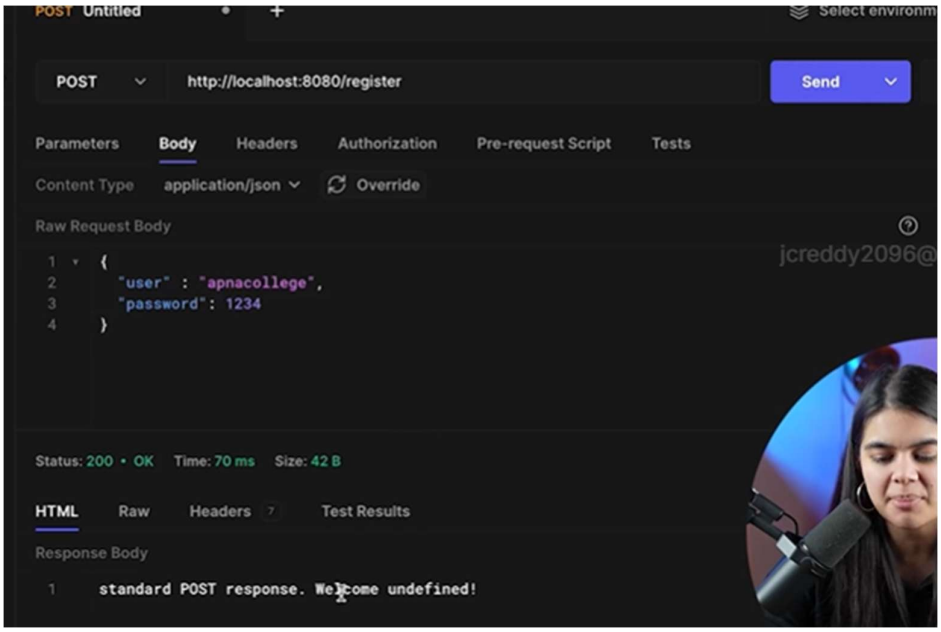

GET & POST Requests
File | /Users/shradhakhapra/WebDevelopment/Backend/Miscellaneous/Frontend/index.html
GET Request Form
enter username
enter password
Submit
POST Request Form
apnacollege
Submit

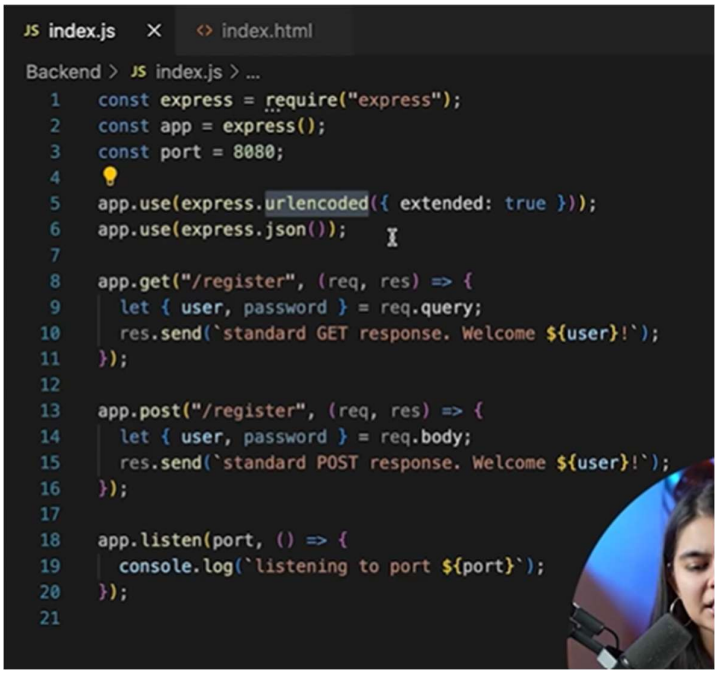

[nodemon] 3.0.1
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
listening to port 8080
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
listening to port 8080
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
listening to port 8080
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
listening to port 8080
undefined
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
listening to port 8080
{ user: 'apnacollege', password: '1234' }
Miscellaneous
index.js
index.html
Backend > index.js > ...
const express = require("express");
const app = express();
const port = 8080;
app.use(express.urlencoded({ extended: true }));
app.get("/register", (req, res) => {
let { user, password } = req.query;
res.send(`standard GET response. Welcome ${user}!`);
});
app.post("/register", (req, res) => {
console.log(req.body);
res.send("standard POST response");
});
app.listen(port, () => {
console.log(`listening to port ${port}`)
});

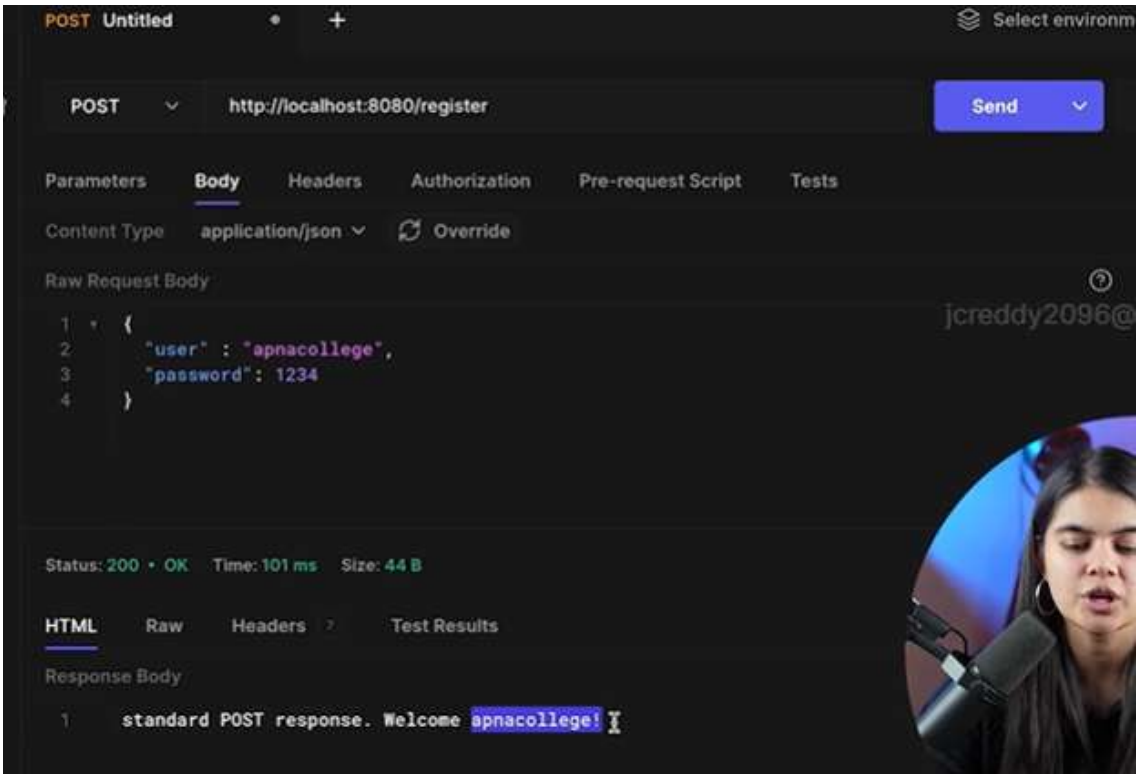

index.js
index.html
Backend > index.js > app.post("/register") callback
1 const express = require("express");
2 const app = express();
3 const port = 8080;
4
5 app.use(express.urlencoded({ extended: true }));
6
7 app.get("/register", (req, res) => {
8 let { user, password } = req.query;
9 res.send(`standard GET response. Welcome ${user}!`);
10 });
11
12 app.post("/register", (req, res) => {
13 let { user, password } = req.body;
14 res.send(`standard POST response. Welcome ${user}!`);
15 });
16
17 app.listen(port, () => {
18 console.log(`listening to port ${port}`);
19 });
20


GET & POST Requests
File | /Users/shradhakhapra/WebDevelopment/Backend/Miscellaneous/Frontend/index.html
GET Request Form
enter username
enter password
Submit
POST Request Form
apnacollege
Submit

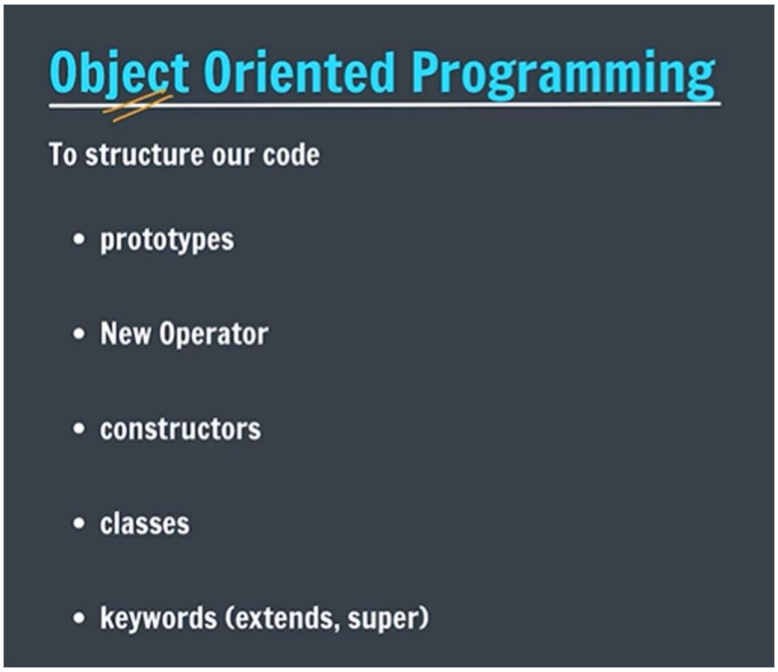
localhost:8080/register
localhost:8080/register
standard POST response. Welcome apnacollege!


POST Untitled
Select environm
POST
http://localhost:8080/register
Send
Parameters
Body
Headers
Authorization
Pre-request Script
Tests
Content Type
application/json
Override
Raw Request Body
jcreddy2096@
{
"user" : "apnacollege",
"password": 1234
}
Status: 200 • OK
Time: 70 ms
Size: 42 B
HTML
Raw
Headers
Test Results
Response Body
standard POST response. Welcome undefined!

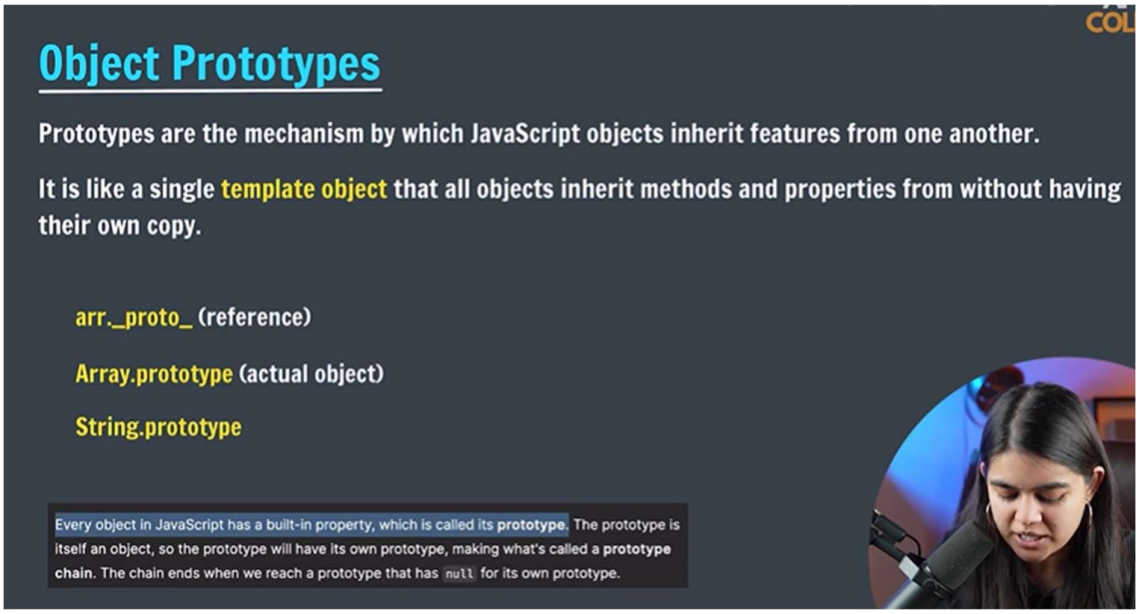

index.js
index.html
Backend > index.js > ...
const express = require("express");
const app = express();
const port = 8080;
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
app.get("/register", (req, res) => {
let { user, password } = req.query;
res.send(`standard GET response. Welcome ${user}!`);
});
app.post("/register", (req, res) => {
let { user, password } = req.body;
res.send(`standard POST response. Welcome ${user}!`);
});
app.listen(port, () => {
console.log(`listening to port ${port}`);
});

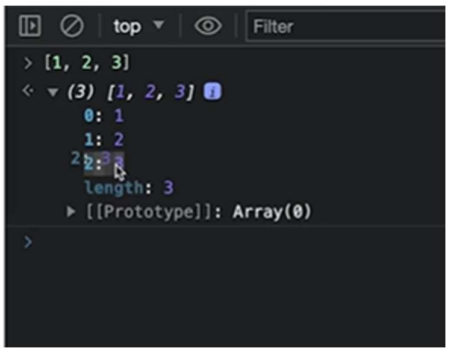

POST Untitled
Select environm
POST
http://localhost:8080/register
Send
Parameters
Body
Headers
Authorization
Pre-request Script
Tests
Content Type
application/json
Override
Raw Request Body
{
"user" : "apnacollege",
"password": 1234
}
jcreddy2096@
Status: 200 • OK
Time: 101 ms
Size: 44 B
HTML
Raw
Headers
Test Results
Response Body
standard POST response. Welcome apnacollege!

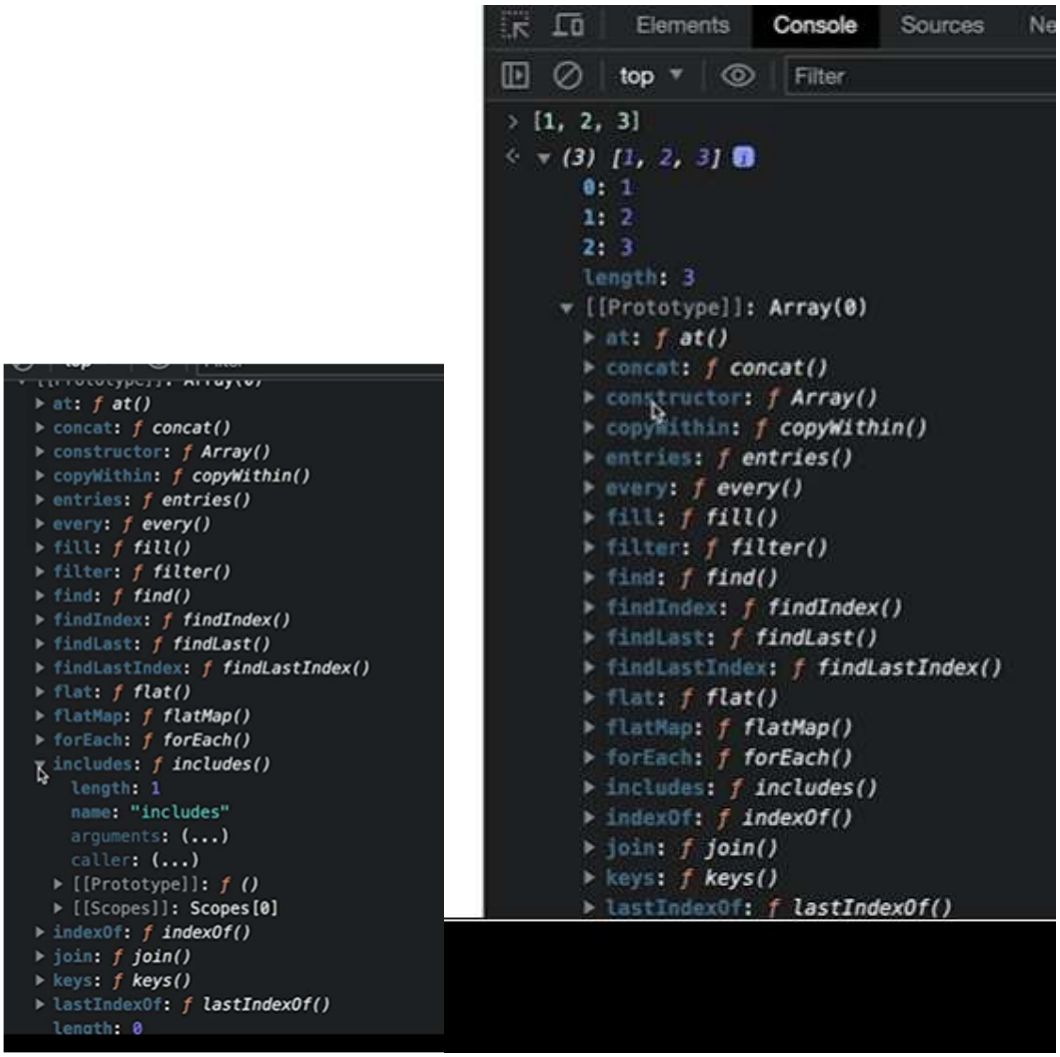
→OOPS


Object Oriented Programming
To structure our code
• prototypes
• New Operator
• constructors
• classes
• keywords (extends, super)

→Object Prototypes



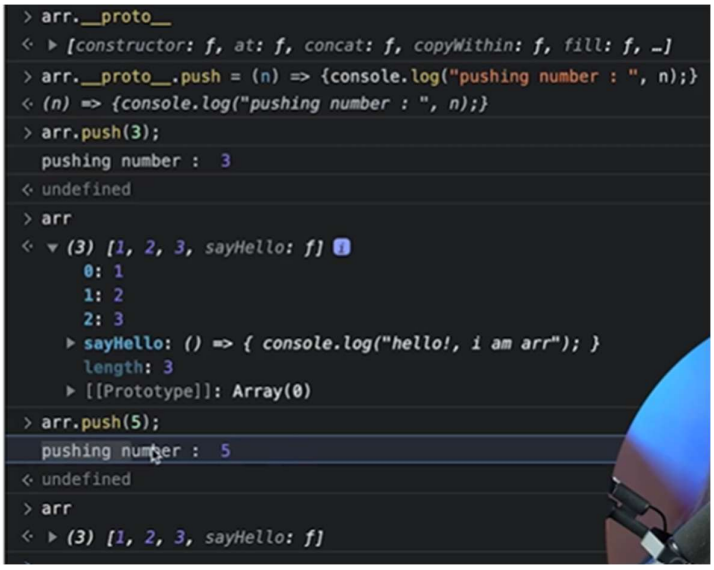


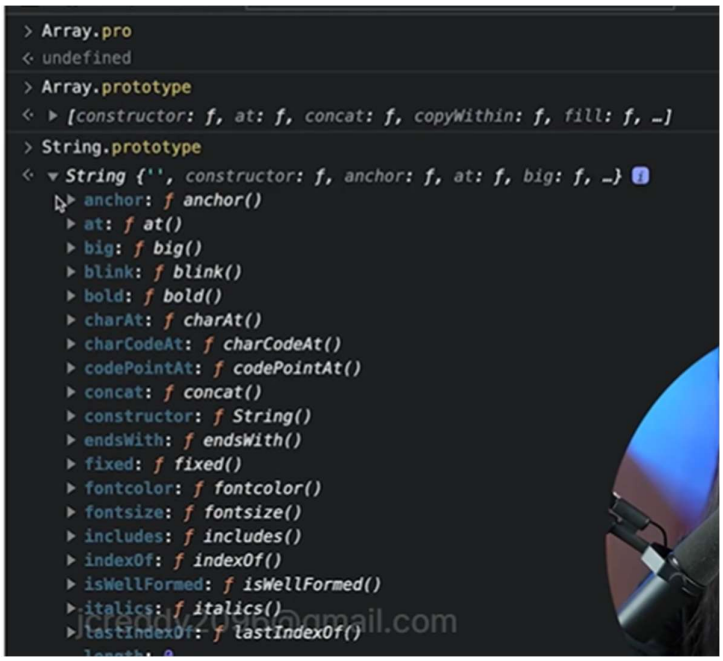

Elements
Console
Sources
top
Filter
> [1, 2, 3]
(3) [1, 2, 3]
0: 1
1: 2
2: 3
length: 3
[[Prototype]]: Array(0)
at: ƒ at()
concat: ƒ concat()
constructor: ƒ Array()
copyWithin: ƒ copyWithin()
entries: ƒ entries()
every: ƒ every()
fill: ƒ fill()
filter: ƒ filter()
find: ƒ find()
findIndex: ƒ findIndex()
findLast: ƒ findLast()
findLastIndex: ƒ findLastIndex()
flat: ƒ flat()
flatMap: ƒ flatMap()
forEach: ƒ forEach()
includes: ƒ includes()
indexOf: ƒ indexOf()
join: ƒ join()
keys: ƒ keys()
lastIndexOf: ƒ lastIndexOf()
length: 0
name: "includes"
arguments: (...)
caller: (...)
[[Prototype]]: ƒ ()
[[Scopes]]: Scopes[0]


index.html
app.js
Frontend > app.js > ...
let arr = [1, 2, 3];
arr.sayHello = () => {
console.log("hello!, i am arr");
};

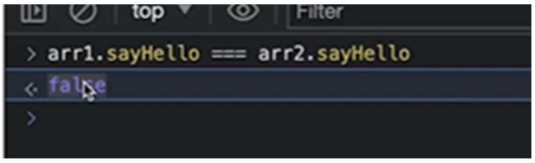
index.html
app.js
Frontend > app.js > ...
let arr = [1, 2, 3];
let arr2 = [1, 2, 3];
arr.sayHello = () => {
console.log("hello!, i am arr");
};
arr2.sayHello = () => {
console.log("hello!, i am arr");
};


> arr.__proto__
[constructor: ƒ, at: ƒ, concat: ƒ, copyWithin: ƒ, fill: ƒ, …]
> arr.__proto__.push = (n) => {console.log("pushing number : ", n);}
(n) => {console.log("pushing number : ", n);}
> arr.push(3);
pushing number : 3
undefined


> arr.__proto__
▶ [constructor: ƒ, at: ƒ, concat: ƒ, copyWithin: ƒ, fill: ƒ, …]
> arr.__proto__.push = (n) => {console.log("pushing number : ", n);}
(n) => {console.log("pushing number : ", n);}
> arr.push(3);
pushing number : 3
undefined
> arr
▼ (3) [1, 2, 3, sayHello: ƒ]
0: 1
1: 2
2: 3
▶ sayHello: () => { console.log("hello!, i am arr"); }
length: 3
▶ [[Prototype]]: Array(0)
> arr.push(5);
pushing number : 5
undefined
> arr
▶ (3) [1, 2, 3, sayHello: ƒ]


> Array.pro
undefined
> Array.prototype
▶ [constructor: ƒ, at: ƒ, concat: ƒ, copyWithin: ƒ, fill: ƒ, …]
> String.prototype
▼ String {'', constructor: ƒ, anchor: ƒ, at: ƒ, big: ƒ, …}
▶ anchor: ƒ anchor()
▶ at: ƒ at()
▶ big: ƒ big()
▶ blink: ƒ blink()
▶ bold: ƒ bold()
▶ charAt: ƒ charAt()
▶ charCodeAt: ƒ charCodeAt()
▶ codePointAt: ƒ codePointAt()
▶ concat: ƒ concat()
▶ constructor: ƒ String()
▶ endsWith: ƒ endsWith()
▶ fixed: ƒ fixed()
▶ fontcolor: ƒ fontcolor()
▶ fontsize: ƒ fontsize()
▶ includes: ƒ includes()
▶ indexOf: ƒ indexOf()
▶ isWellFormed: ƒ isWellFormed()
▶ italics: ƒ italics()
▶ lastIndexOf: ƒ lastIndexOf()


index.html
app.js
Frontend > app.js > ...
1 let arr1 = [1, 2, 3];
2 let arr2 = [1, 2, 3];
3
4 arr1.sayHello = () => {
5 console.log("hello!, i am arr");
6 };
7
8 arr2.sayHello = () => {
9 console.log("hello!, i am arr");
10 };
11


top
Filter
> arr1.sayHello === arr2.sayHello
false

```
> arr1.sayHello === arr2.sayHello
< false
> "abc".toUpperCase === "xyz".toUpperCase
< true
>
```

→Factory Functions


```
index.html   app.js
Frontend > app.js > PersonMaker > person
1  function PersonMaker(name, age) {
2    const person = {
3      name: name,
4      age: age,
5    };
6
7    return person;
8  }
```

```
function PersonMaker(name, age) {
  const person = {
    name: name,
    age: age,
    talk() {
      console.log(`Hi, my name is ${this.name}`);
    },
  };

  return person;
}
```

```
> let p1 = PersonMaker("adam", 25);
< undefined
> p1
< {name: 'adam', age: 25, talk: f}
    age: 25
    name: "adam"
  ▶ talk: f talk()
  ▶ [[Prototype]]: Object
> p1.talk();
  Hi, my name is adam
< undefined
> let p2 = PersonMaker("eve", 25);
< undefined
> p2
< {name: 'eve', age: 25, talk: f}
> p2.talk();
  Hi, my name is eve
< undefined
```

```
index.html   app.js
Frontend > app.js > ...
1  function PersonMaker(name, age) {
2    const person = {
3      name: name,
4      age: age,
5      talk() {
6        console.log(`Hi, my name is ${this.name}`);
7      },
8    };
9
10   return person;
11 }
12
13 let p1 = PersonMaker("adam", 25); //copy
14 let p2 = PersonMaker("eve", 25);//copy
15
```

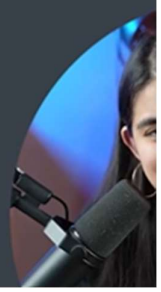
New operator

The new operator lets developers create an instance of a user-defined object type or of one of the built-in object types that has a constructor function.

```
function Person(name, age) {
  this.name = name;
  this.age = age;
}

Person.prototype.talk = function () {
  console.log(`Hi, my name is ${this.name}`);
};

let p1 = new Person("adam", 25);
let p2 = new Person("eve", 25);
```



Frontend > JS app.js > ...

```
10  //   return person;
11  // }
12
13  // Constructors - doesn't return anything & start with capital
14  function Person(name, age) {
15    this.name = name;
16    this.age = age;
17    console.log(this);
18  }
19
20  Person.prototype.talk = function () {
21    console.log(`Hi, ,my name is ${this.name}`);
22  };
23
24  let p1 = new Person("adam", 25);
25  let p2 = new Person("eve", 25);
26
```

Classes

Classes are a **template** for creating objects

The **constructor** method is a special method of a class for creating and initializing an object instance of that class.

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  talk() {
    console.log(`Hi, my name is ${this.name}`);
  }
}

let p1 = new Person("adam", 25);
let p2 = new Person("eve", 25);
```

```
class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }
  talk() {
    console.log(`Hi, ,my name is ${this.name}`);
  }
}

let p1 = new Person("adam", 25);
let p2 = new Person("eve", 25);
```



```
> p1
< ▼ Person {name: 'adam', age: 25}
    age: 25
    name: "adam"
  ▶ [[Prototype]]: Object
> p2
< ▼ Person {name: 'eve', age: 25}
    age: 25
    name: "eve"
  ▼ [[Prototype]]: Object
    ▶ constructor: class Person
    ▶ talk: ƒ talk()
    ▶ [[Prototype]]: Object
>
```

```
> p1
< ▼ Person {name: 'adam', age: 25}
    age: 25
    name: "adam"
  ▶ [[Prototype]]: Object
> p2
< ▶ Person {name: 'eve', age: 25}
> p1.talk === p2.talk
< true
>
```

Inheritance

Inheritance is a mechanism that allows us to create new classes on the basis of already existing classes.

```
class Student extends Person {
  constructor(name, age, marks) {
    super(name, age);
    this.marks = marks;
  }

  greet() {
    return "hello!";
  }
}

let s1 = new Student("adam", 25, 95);
```

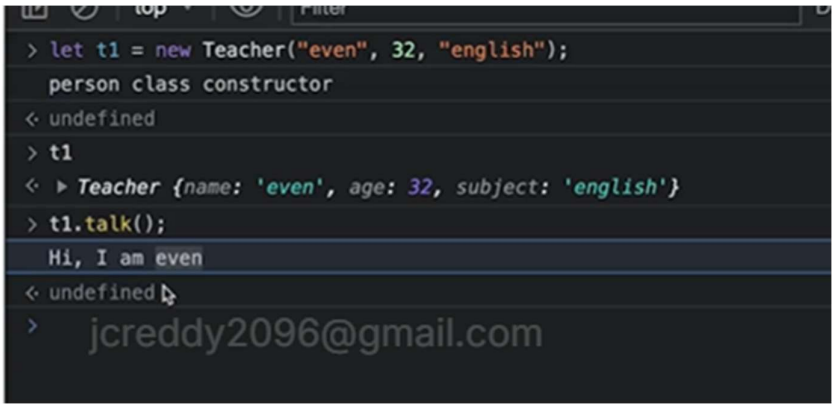
index.html app.js

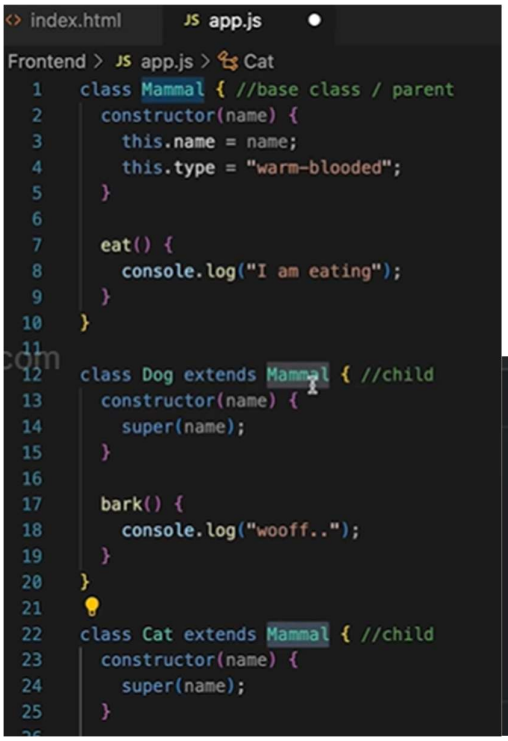
Frontend > app.js > Person > constructor

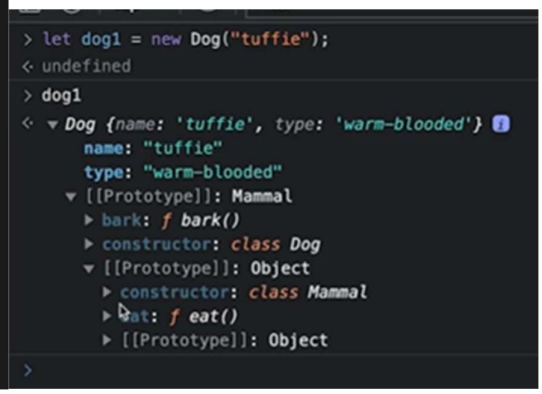
```
1   class Person {
2     constructor(name, age) {
3       console.log("person class constructor");
4       this.name = name;
5       this.age = age;
6     }
7     talk() {
8       console.log(`Hi, I am ${this.name}`);
9     }
10  }
11
12  class Student extends Person {
13    constructor(name, age, marks) {
14      console.log("student class constructor");
15      super(name, age); //parent class constructor is being called
16      this.marks = marks;
17    }
18  }
19
20  class Teacher extends Person {
21    constructor(name, age, subject) {
22      super(name, age); //parent class constructor is being cal
23      this.subject = subject;
24    }
25  }
```

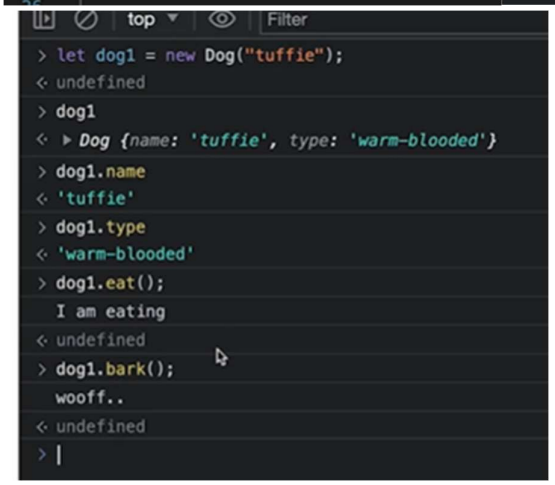
```
> let stu1 = new Student("adam", 25, 95);
  student class constructor
  person class constructor
< undefined
> stu1.marks
< 95
> stu1.name
< 'adam'
> stu1.age
< 25
> stu1.talk();
  Hi, I am adam
< undefined
>
```

```
> let t1 = new Teacher("even", 32, "english");
  person class constructor
< undefined
> t1
< ▼ Teacher {name: 'even', age: 32, subject: 'english'}
    age: 32
    name: "even"
    subject: "english"
  ▼ [[Prototype]]: Person
    ▶ constructor: class Teacher
    ▼ [[Prototype]]: Object
      ▶ constructor: class Person
      ▶ talk: ƒ talk()
      ▶ [[Prototype]]: Object
>
```

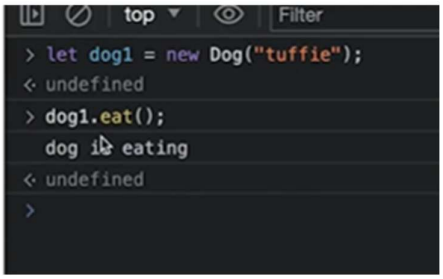

top
Filter
> let t1 = new Teacher("even", 32, "english");
person class constructor
undefined
> t1
▶ Teacher {name: 'even', age: 32, subject: 'english'}
> t1.talk();
Hi, I am even
undefined
jcreddy2096@gmail.com


index.html
app.js
Frontend > app.js > Cat
1 class Mammal { //base class / parent
2 constructor(name) {
3 this.name = name;
4 this.type = "warm-blooded";
5 }
6
7 eat() {
8 console.log("I am eating");
9 }
10 }
11
12 class Dog extends Mammal { //child
13 constructor(name) {
14 super(name);
15 }
16
17 bark() {
18 console.log("wooff..");
19 }
20 }
21
22 class Cat extends Mammal { //child
23 constructor(name) {
24 super(name);
25 }


> let dog1 = new Dog("tuffie");
undefined
> dog1
▼ Dog {name: 'tuffie', type: 'warm-blooded'}
name: "tuffie"
type: "warm-blooded"
▼ [[Prototype]]: Mammal
▶ bark: ƒ bark()
▶ constructor: class Dog
▼ [[Prototype]]: Object
▶ constructor: class Mammal
▶ eat: ƒ eat()
▶ [[Prototype]]: Object


top
Filter
> let dog1 = new Dog("tuffie");
undefined
> dog1
▶ Dog {name: 'tuffie', type: 'warm-blooded'}
> dog1.name
'tuffie'
> dog1.type
'warm-blooded'
> dog1.eat();
I am eating
undefined
> dog1.bark();
wooff..
undefined


index.html
app.js
Frontend > app.js > Dog > eat
this.type = "warm-blooded";
}
eat() {
console.log("I am eating");
}
}
class Dog extends Mammal {
//child
constructor(name) {
super(name);
}
bark() {
console.log("wooff..");
}
eat() {
console.log("dog is eating");
}
}
class Cat extends Mammal {
//child
constructor(name) {
jcreddy2096@gmail.com


top
Filter
let dog1 = new Dog("tuffie");
undefined
dog1.eat();
dog is eating
undefined